

Energy Demand Analysis and Forecasting

Author: Adam Lawit

Objective: Create and evaluate multiple forecasting approaches to predict electrical demand over an adjustable period using hourly CAISO data.

The goal is to:

- Capture daily seasonality (24-hour cycle)
- Improve forecast accuracy using time-based predictors
- Evaluate model performance using MAE
- Analyze forecast errors by hour of day

Dataset:

- Source: CAISO (California Independent System Operator)
- Resolution: Hourly demand data
- Period: Jan 1, 2024 – Dec 31, 2025
- Features: Timestamp, Demand (MW), etc

In [426...

```
# 1. Imports
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.metrics import mean_absolute_error, mean_squared_error
from statsmodels.tsa.statespace.sarimax import SARIMAX
from statsmodels.tsa.seasonal import seasonal_decompose
from prophet import Prophet
```

In [427...

```
# 2. Load & Prepare Data

hourly = pd.read_csv(
    "EnergyData/Hourly_CAISO_2425.csv",
    parse_dates=["Local time"],
    thousands=",",
)
hourly = hourly.sort_values("Local time").set_index("Local time")
```

In [428...

```
# 3a. Quick Data Quality Checks

print("Missing values for selected columns:\n", hourly[["Demand"]].isna().sum())
print("Duplicate timestamps:", hourly.index.duplicated().sum())
```

Missing values for selected columns:

Demand 96

dtype: int64

Duplicate timestamps: 2

```
In [429... # 3b. Handle Missing and Duplicates
# Interpolate missing demand values
hourly["Demand"] = hourly["Demand"].interpolate(method="time")
#Use Mean of Duplicates
hourly = hourly.groupby(hourly.index).median(numeric_only=True)
# Verify
print("Missing values after interpolation:", hourly["Demand"].isna().sum())
print("Duplicate timestamps after aggregation:", hourly.index.duplicated().sum())
```

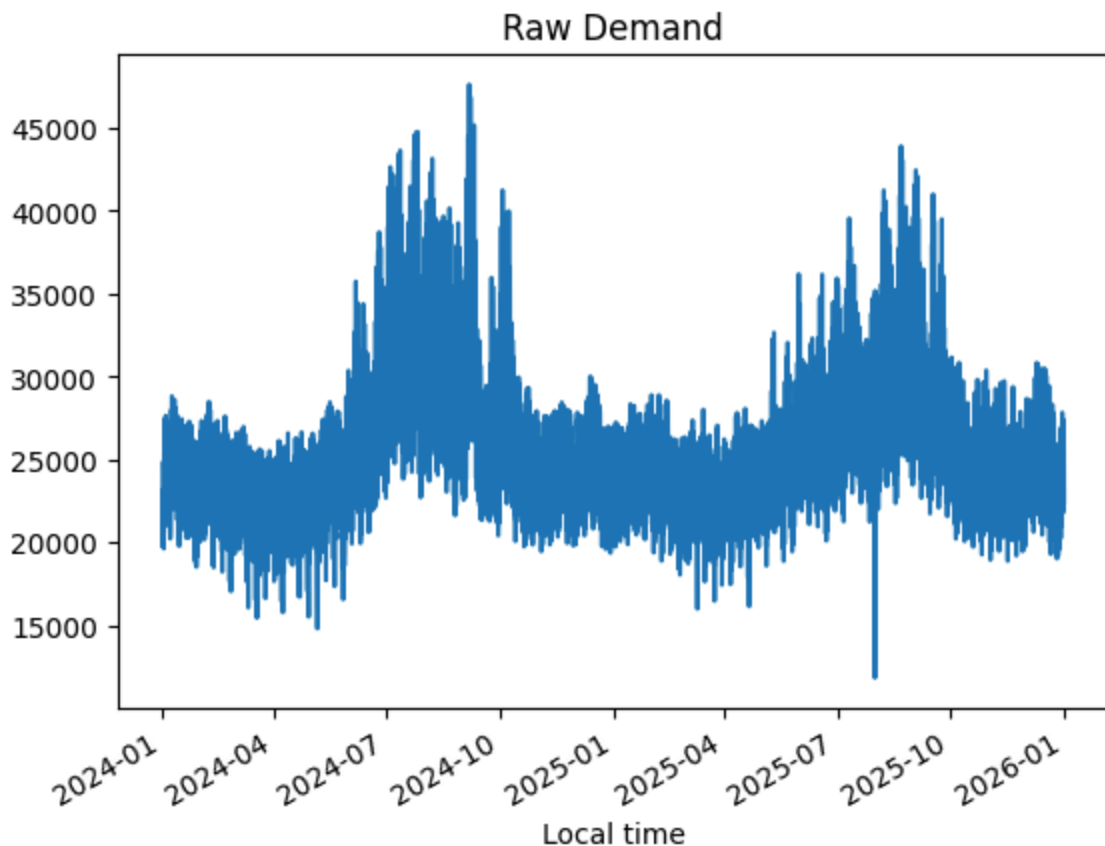
Missing values after interpolation: 0

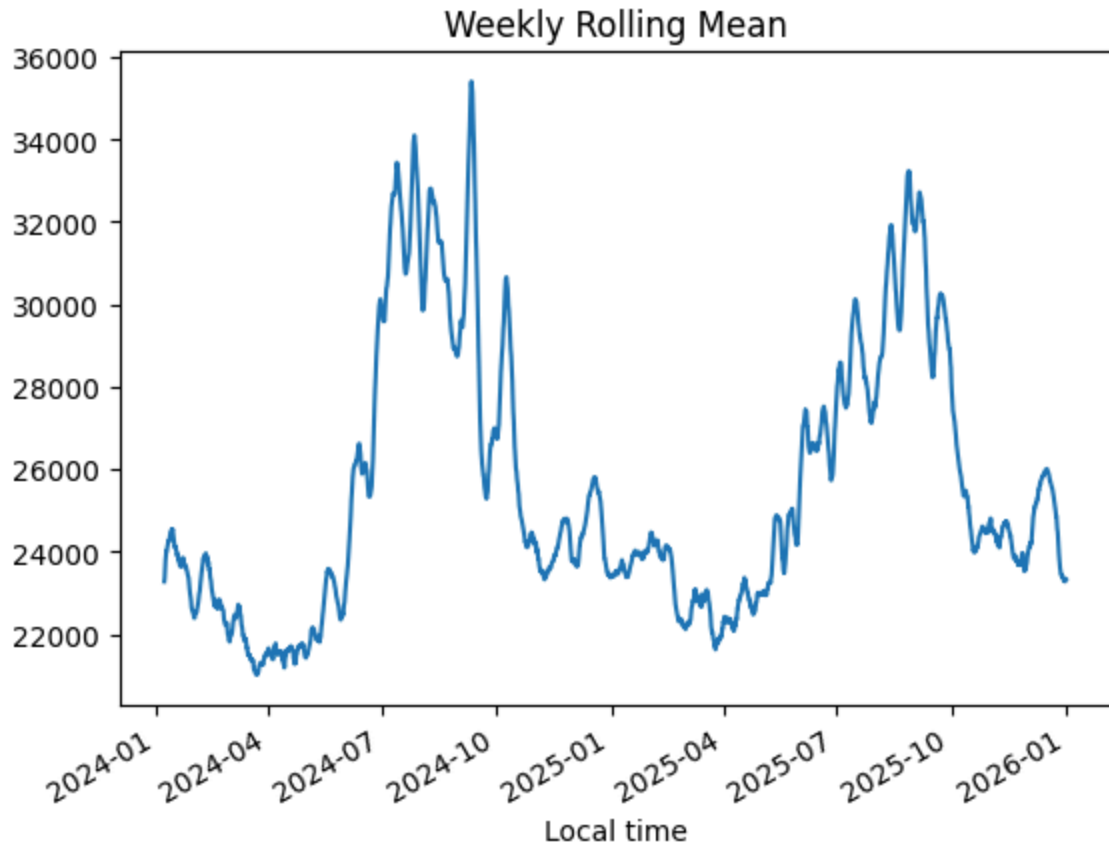
Duplicate timestamps after aggregation: 0

```
In [430... #3c. Plot

hourly["Demand"].plot(title="Raw Demand")
plt.show()

# Rolling trend
hourly["Demand"].rolling(24*7).mean().plot(title="Weekly Rolling Mean")
plt.show()
```





In [431...

4. Feature Engineering

```
hourly["hour"] = hourly.index.hour
hourly["dayofweek"] = hourly.index.dayofweek
hourly["month"] = hourly.index.month
hourly["is_weekend"] = (hourly["dayofweek"] >= 5).astype(int) # 5 = Saturday, 6 = Sunday

hourly["is_month_start"] = hourly.index.is_month_start.astype(int)
hourly["is_month_end"] = hourly.index.is_month_end.astype(int)
hourly["quarter"] = hourly.index.quarter
hourly["day_of_year"] = hourly.index.dayofyear
```

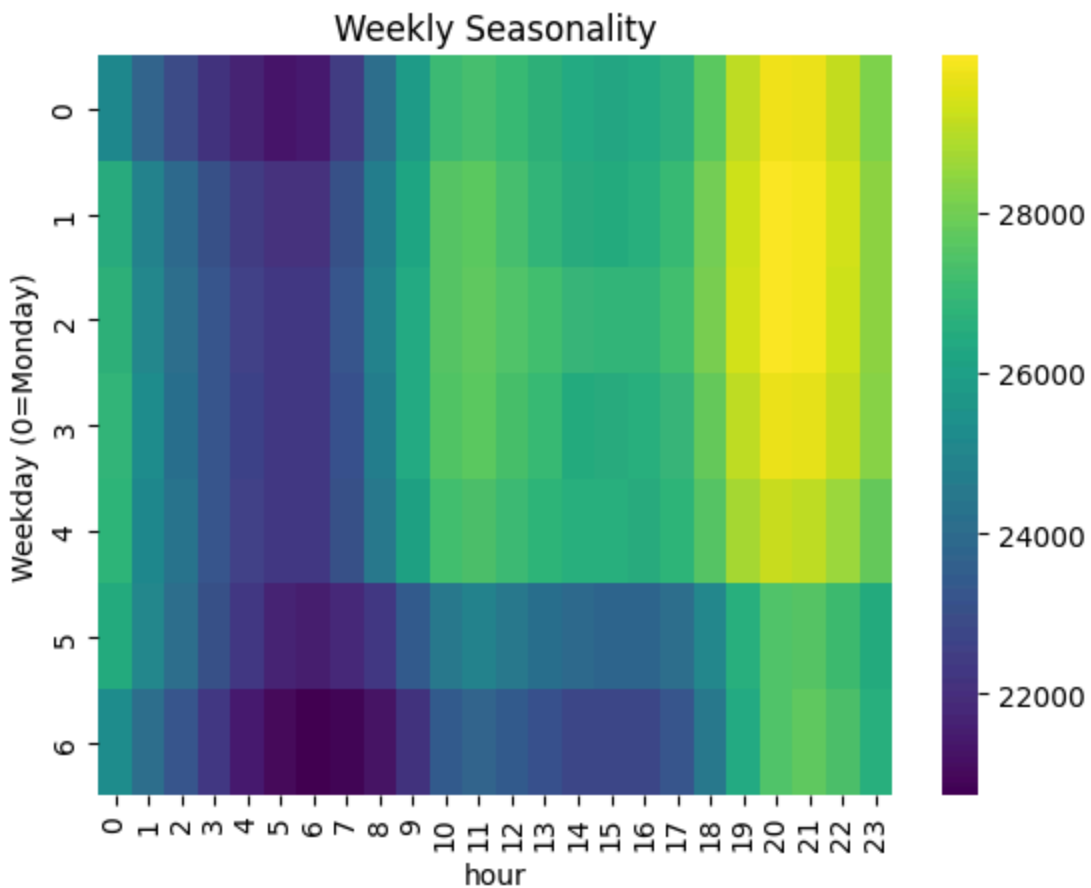
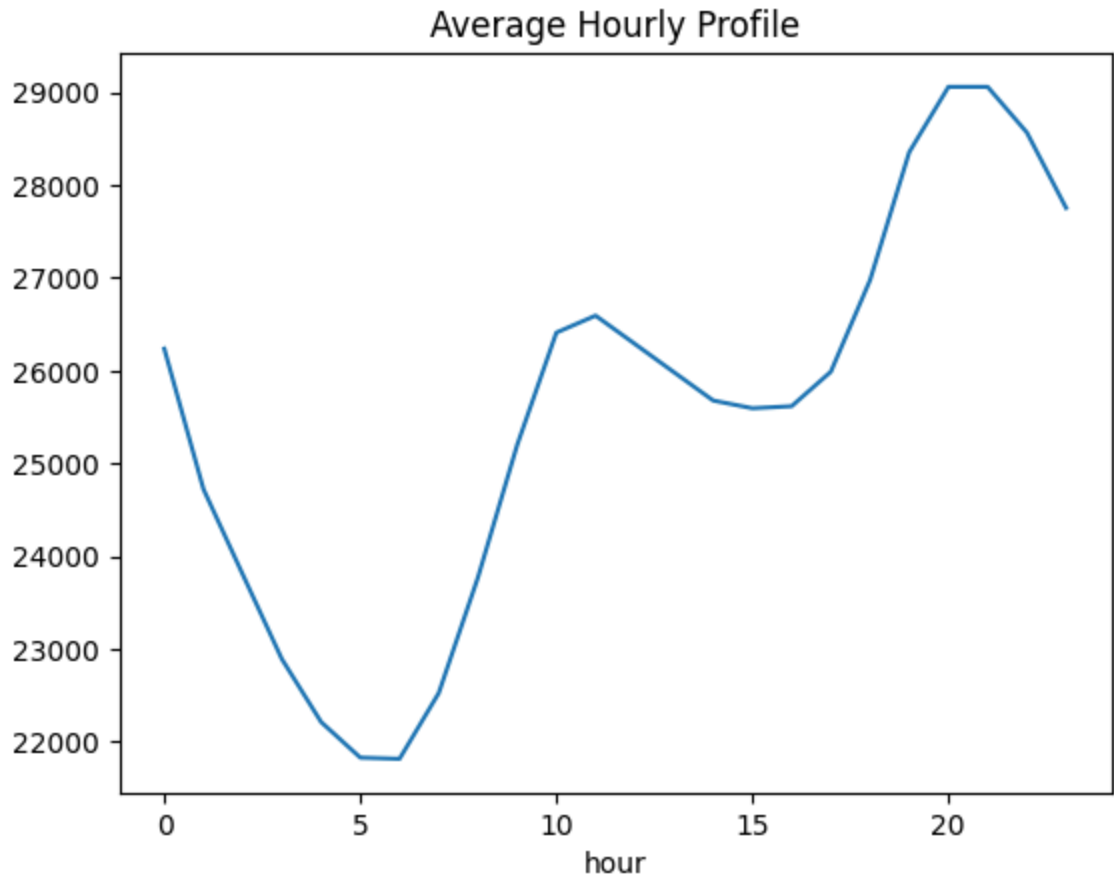
In [432...

5. Exploratory Analysis

```
# Average hourly demand
hourly.groupby("hour")["Demand"].mean().plot(title="Average Hourly Profile")
plt.show()

# Day/hour heatmap
pivot = hourly.pivot_table(values="Demand",
                             index="dayofweek",
                             columns="hour",
                             aggfunc="mean")

sns.heatmap(pivot, cmap='viridis')
plt.title("Weekly Seasonality")
plt.ylabel("Weekday (0=Monday)")
plt.show()
```



```

In [433... # Define the adjustable evaluation period
eval_start = pd.Timestamp("2025-3-12 00:00:00") # start of evaluation
eval_end    = pd.Timestamp("2025-3-25 23:00:00") # end of evaluation

In [434... # Yesterday Naive Baseline Forecast

hourly["baseline_yesterday"] = hourly["Demand"].shift(24)

# Evaluation data
baseline_eval = hourly.loc[
    (hourly.index >= eval_start) &
    (hourly.index <= eval_end) &
    hourly["baseline_yesterday"].notna()
].copy()

# Standard columns used everywhere
baseline_eval["actual"] = baseline_eval["Demand"]
baseline_eval["forecast"] = baseline_eval["baseline_yesterday"]

# Absolute error
baseline_eval["abs_error"] = (
    baseline_eval["actual"] - baseline_eval["forecast"]
).abs()

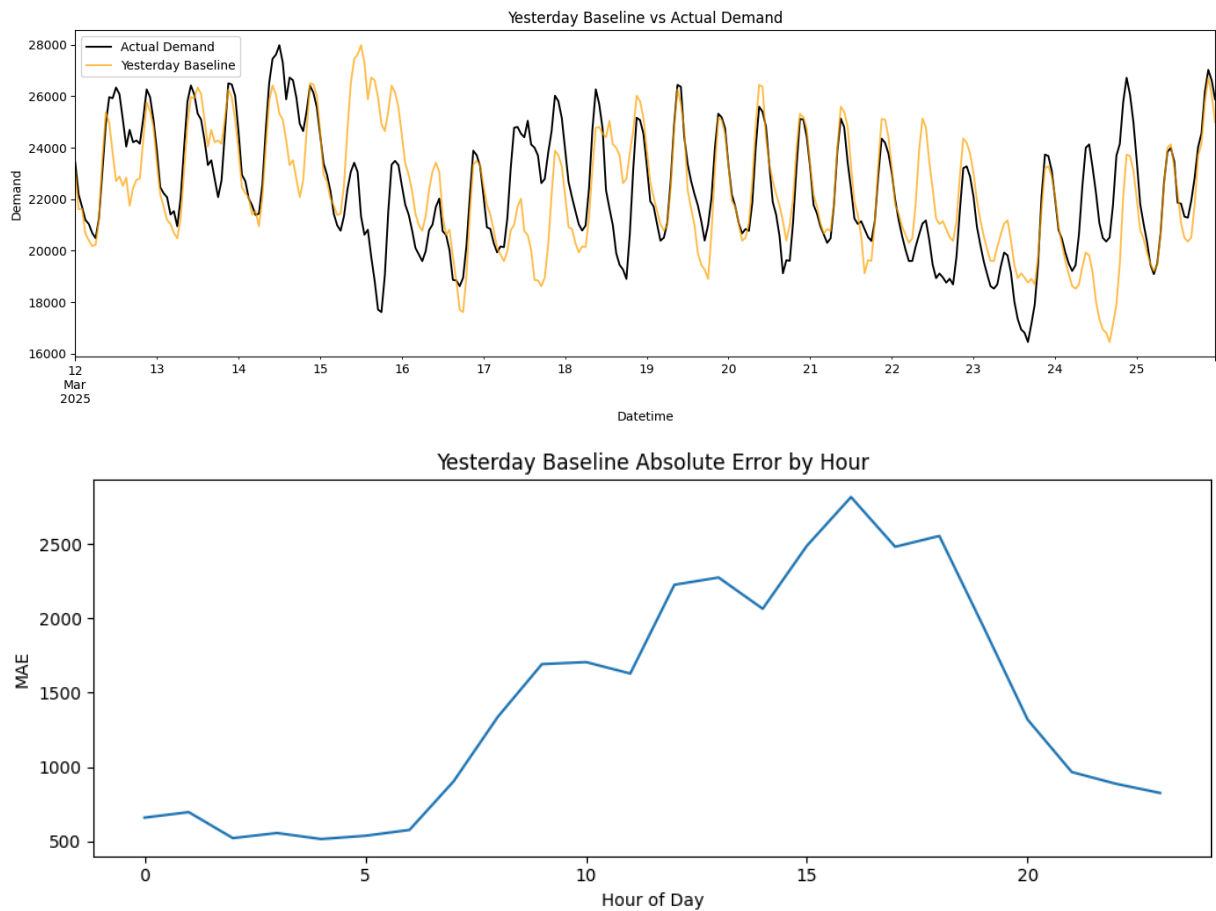
# MAE
baseline_mae = baseline_eval["abs_error"].mean()
print(f"Yesterday Baseline MAE: {baseline_mae:.2f}")

# Plot
plt.figure(figsize=(14,5))
baseline_eval["actual"].plot(label="Actual Demand", color="black")
baseline_eval["forecast"].plot(label="Yesterday Baseline", color="orange", alpha=0.5)
plt.title("Yesterday Baseline vs Actual Demand")
plt.xlabel("Datetime")
plt.ylabel("Demand")
plt.legend()
plt.tight_layout()
plt.show()

# Hourly MAE
plt.figure(figsize=(10,4))
baseline_eval.groupby(baseline_eval.index.hour)["abs_error"].mean().plot()
plt.title("Yesterday Baseline Absolute Error by Hour")
plt.xlabel("Hour of Day")
plt.ylabel("MAE")
plt.tight_layout()
plt.show()

```

Yesterday Baseline MAE: 1424.32



In [435...

```
# Last Week Naive Baseline

# Create forecast
hourly["baseline_lastweek"] = hourly["Demand"].shift(24 * 7)

# Evaluation slice
lastweek_eval = hourly.loc[
    (hourly.index >= eval_start) &
    (hourly.index <= eval_end) &
    hourly["baseline_lastweek"].notna()
].copy()

# Standard columns
lastweek_eval["actual"] = lastweek_eval["Demand"]
lastweek_eval["forecast"] = lastweek_eval["baseline_lastweek"]

# Error
lastweek_eval["abs_error"] = (
    lastweek_eval["actual"] - lastweek_eval["forecast"]
).abs()

# MAE
lastweek_mae = lastweek_eval["abs_error"].mean()
print(f"Last-Week Baseline MAE: {lastweek_mae:.2f}")

# Plot
plt.figure(figsize=(14,5))
lastweek_eval["actual"].plot(label="Actual", color="black")
```

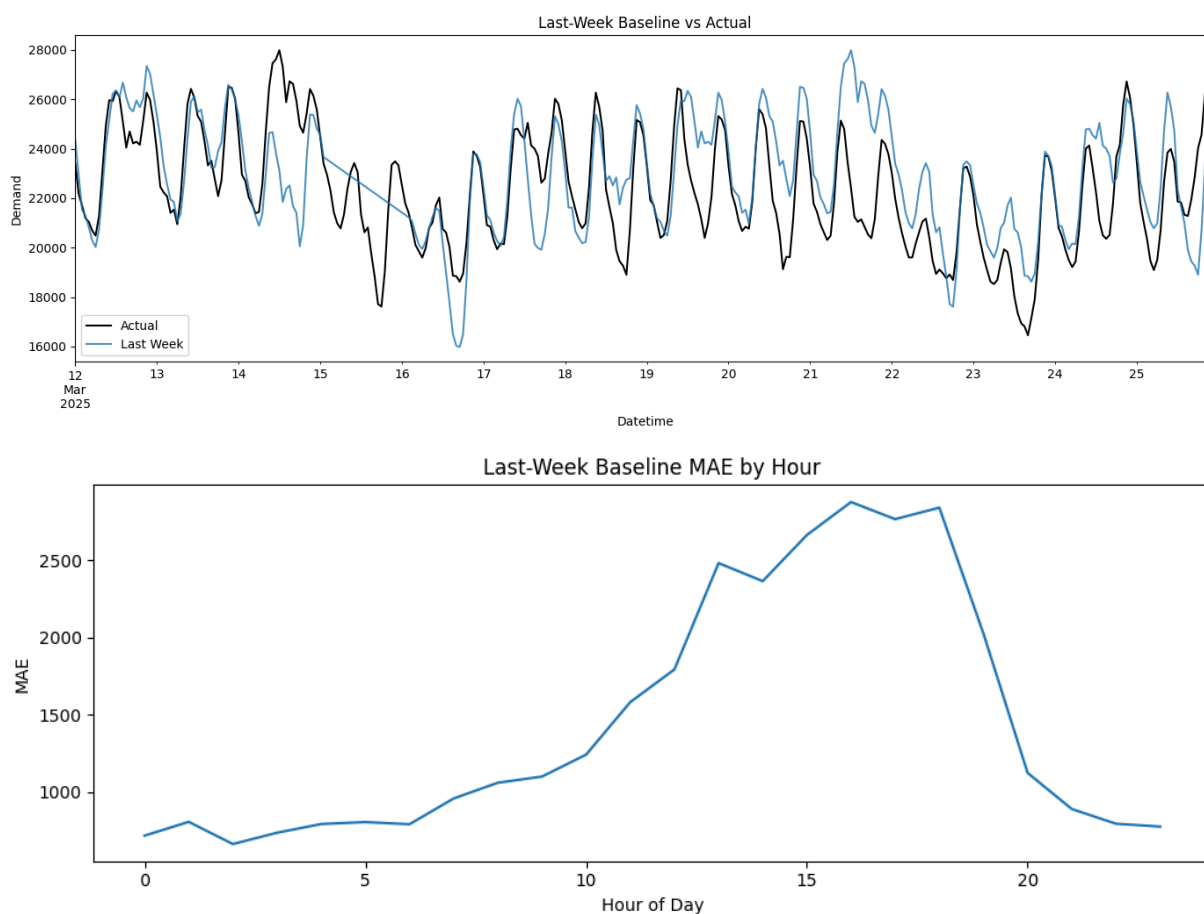
```

lastweek_eval["forecast"].plot(label="Last Week", alpha=0.8)
plt.title("Last-Week Baseline vs Actual")
plt.xlabel("Datetime")
plt.ylabel("Demand")
plt.legend()
plt.tight_layout()
plt.show()

# Hourly MAE
plt.figure(figsize=(10,4))
lastweek_eval.groupby(lastweek_eval.index.hour)["abs_error"].mean().plot()
plt.title("Last-Week Baseline MAE by Hour")
plt.xlabel("Hour of Day")
plt.ylabel("MAE")
plt.tight_layout()
plt.show()

```

Last-Week Baseline MAE: 1443.46



In [441...

```

# SARIMAX Forecast

# Training months
eval_months = hourly.loc[eval_start:eval_end].index.month.unique()

train_data = hourly.loc[
    (hourly.index < eval_start) &
    (hourly.index.month.isin(eval_months)),
    "Demand"
]

```

```
# Exogenous variables
train_exog = pd.DataFrame({
    "hour": train_data.index.hour,
    "dayofweek": train_data.index.dayofweek
}, index=train_data.index)

# Model
sarimax_model = SARIMAX(
    train_data,
    exog=train_exog,
    order=(1,0,1),
    seasonal_order=(1,1,1,24),
    trend='n',
    enforce_stationarity=False,
    enforce_invertibility=False
)

sarimax_result = sarimax_model.fit(dispatch=False)

# Forecast for evaluation window
eval_index = hourly.loc[eval_start:eval_end].index
steps = len(eval_index)

eval_exog = pd.DataFrame({
    "hour": eval_index.hour,
    "dayofweek": eval_index.dayofweek
}, index=eval_index)

forecast_obj = sarimax_result.get_forecast(
    steps=steps,
    exog=eval_exog
)

sarima_eval = pd.DataFrame({
    "actual": hourly.loc[eval_start:eval_end, "Demand"],
    "forecast": forecast_obj.predicted_mean.values
}, index=eval_index)

# Error
sarima_eval["abs_error"] = (
    sarima_eval["actual"] - sarima_eval["forecast"]
).abs()

sarima_mae = mean_absolute_error(
    sarima_eval["actual"],
    sarima_eval["forecast"]
)

print(f"SARIMAX MAE: {sarima_mae:.2f}")

# Plot
plt.figure(figsize=(14,5))
sarima_eval["actual"].plot(label="Actual Demand", color="black")
sarima_eval["forecast"].plot(label="SARIMAX Forecast", color="red")
plt.title("SARIMAX Forecast vs Actual Demand")
```

```
plt.xlabel("Datetime")
plt.ylabel("Demand")
plt.legend()
plt.tight_layout()
plt.show()

# Hourly MAE
plt.figure(figsize=(10,4))
sarima_eval.groupby(sarima_eval.index.hour)["abs_error"].mean().plot()
plt.title("SARIMAX Absolute Error by Hour")
plt.xlabel("Hour of Day")
plt.ylabel("MAE")
plt.tight_layout()
plt.show()
```

```
c:\Users\adaml\AppData\Local\Programs\Python\Python312\Lib\site-packages\statsmodels
\tsa\base\tsa_model.py:473: ValueWarning: A date index has been provided, but it has
no associated frequency information and so will be ignored when e.g. forecasting.
```

```
self._init_dates(dates, freq)
```

```
c:\Users\adaml\AppData\Local\Programs\Python\Python312\Lib\site-packages\statsmodels
\tsa\base\tsa_model.py:473: ValueWarning: A date index has been provided, but it has
no associated frequency information and so will be ignored when e.g. forecasting.
```

```
self._init_dates(dates, freq)
```

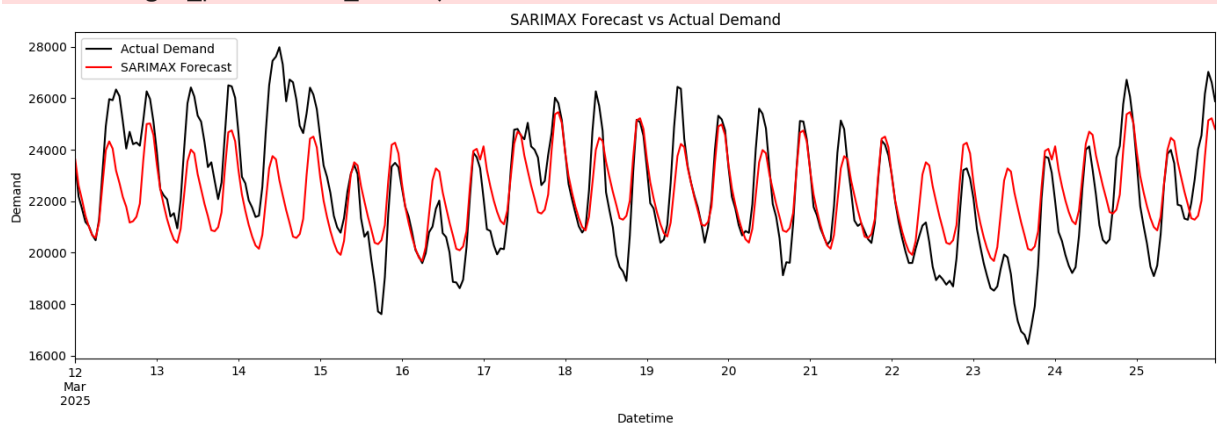
```
c:\Users\adaml\AppData\Local\Programs\Python\Python312\Lib\site-packages\statsmodels
\base\model.py:607: ConvergenceWarning: Maximum Likelihood optimization failed to co
nverge. Check mle_retvals
```

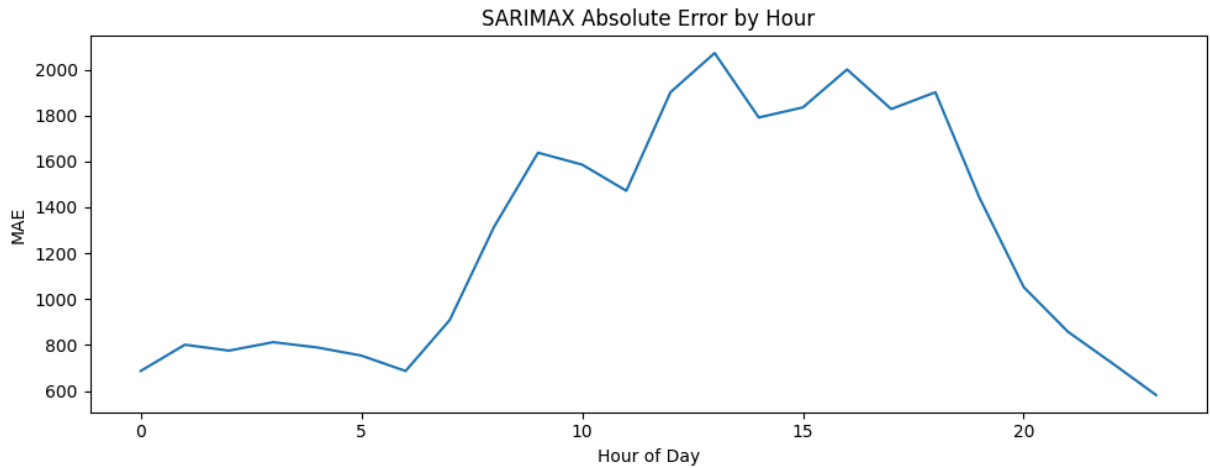
```
warnings.warn("Maximum Likelihood optimization failed to "
```

```
SARIMAX MAE: 1258.77
```

```
c:\Users\adaml\AppData\Local\Programs\Python\Python312\Lib\site-packages\statsmodels
\tsa\base\tsa_model.py:837: ValueWarning: No supported index is available. Predictio
n results will be given with an integer index beginning at `start`.
```

```
return get_prediction_index(
```





In [437...

```
# Prophet Forecast
eval_months = hourly.loc[eval_start:eval_end].index.month.unique()

train_df = hourly.loc[
    (hourly.index < eval_start) &
    (hourly.index.month.isin(eval_months)),
    "Demand"
].copy().reset_index()

print("Training months used:", eval_months)

train_df.columns = ["ds", "y"]
train_df["ds"] = pd.to_datetime(train_df["ds"])
train_df["y"] = train_df["y"].astype(float)

# Extra regressors
train_df["hour"] = train_df["ds"].dt.hour
train_df["dayofweek"] = train_df["ds"].dt.dayofweek
train_df["is_weekend"] = (train_df["dayofweek"] >= 5).astype(int)

# Model
prophet_model = Prophet(
    daily_seasonality=True,
    weekly_seasonality=True,
    yearly_seasonality=True,
    changepoint_prior_scale=.75,
    seasonality_prior_scale=10
)

prophet_model.add_regressor("hour")
prophet_model.add_regressor("dayofweek")
prophet_model.add_regressor("is_weekend")

prophet_model.fit(train_df)

# Forecast
eval_index = hourly.loc[eval_start:eval_end].index

future_df = pd.DataFrame({"ds": eval_index})
future_df["hour"] = future_df["ds"].dt.hour
```

```

future_df["dayofweek"] = future_df["ds"].dt.dayofweek
future_df["is_weekend"] = (future_df["dayofweek"] >= 5).astype(int)

forecast = prophet_model.predict(future_df)

# Force alignment by datetime
forecast = forecast.set_index("ds").reindex(eval_index)

prophet_eval = pd.DataFrame(index=eval_index)
prophet_eval["actual"] = hourly.loc[eval_index, "Demand"]
prophet_eval["forecast"] = forecast["yhat"]

# Error
prophet_eval["abs_error"] = (
    prophet_eval["actual"] - prophet_eval["forecast"]
).abs()

prophet_mae = prophet_eval["abs_error"].mean()

print(f"Prophet MAE: {prophet_mae:.2f}")

# Plot
plt.figure(figsize=(14,5))
prophet_eval["actual"].plot(label="Actual Demand", color="black")
prophet_eval["forecast"].plot(label="Prophet Forecast", color="blue")
plt.title("Prophet Forecast vs Actual Demand")
plt.xlabel("Datetime")
plt.ylabel("Demand")
plt.legend()
plt.tight_layout()
plt.show()

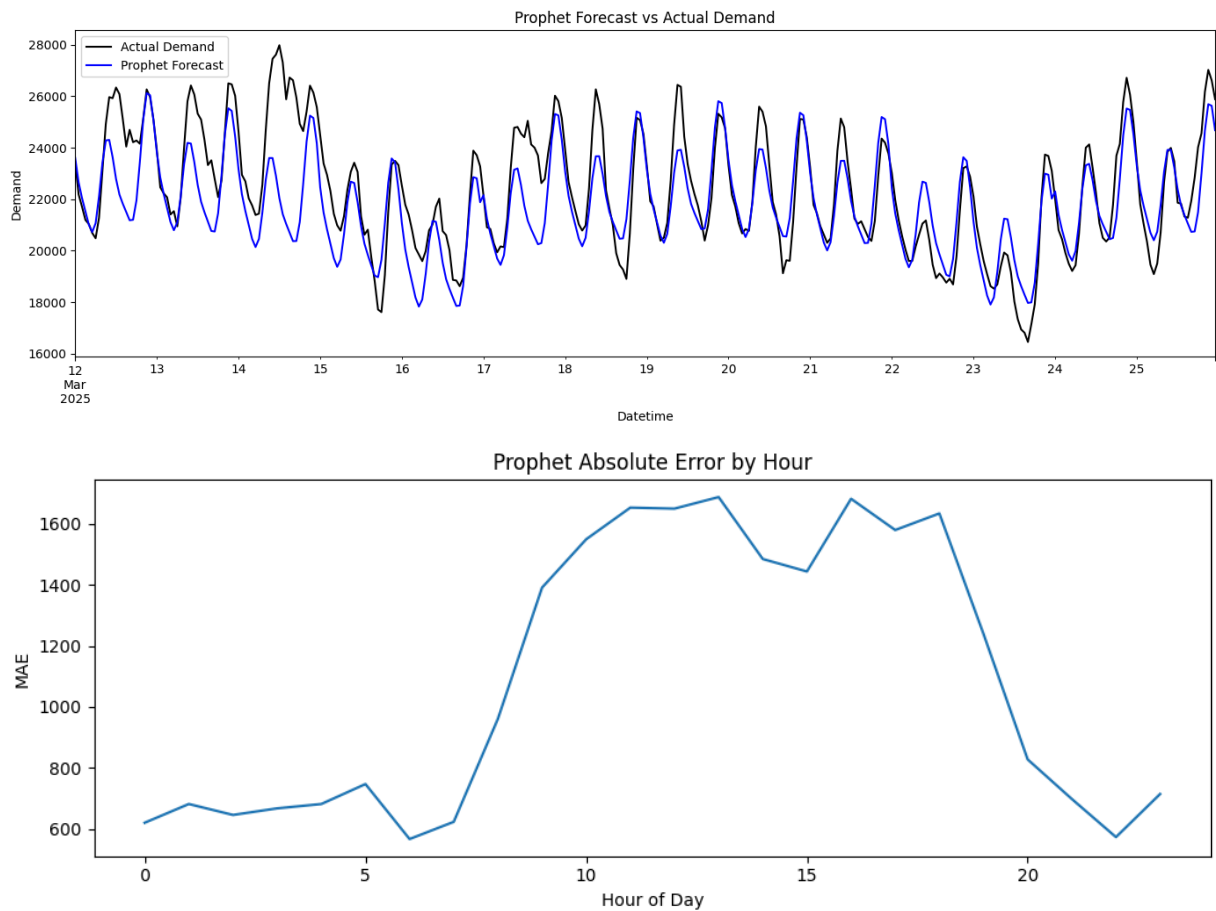
# Hourly MAE
plt.figure(figsize=(10,4))
prophet_eval.groupby(prophet_eval.index.hour)["abs_error"].mean().plot()
plt.title("Prophet Absolute Error by Hour")
plt.xlabel("Hour of Day")
plt.ylabel("MAE")
plt.tight_layout()
plt.show()

```

```

20:47:23 - cmdstanpy - INFO - Chain [1] start processing
Training months used: Index([3], dtype='int32', name='Local time')
20:47:24 - cmdstanpy - INFO - Chain [1] done processing
Prophet MAE: 1083.35

```



In [438...]

```
# Hybrid = Prophet + Last Week

# Align indices just in case
common_index = prophet_eval.index.intersection(lastweek_eval.index)

hybrid_eval = pd.DataFrame(index=common_index)

hybrid_eval["actual"] = prophet_eval.loc[common_index, "actual"]
hybrid_eval["prophet"] = prophet_eval.loc[common_index, "forecast"]
hybrid_eval["lastweek"] = lastweek_eval.loc[common_index, "forecast"]

# CHANGE WEIGHTS HERE
w_prophet = 0.65
w_lastweek = 0.35

hybrid_eval["forecast"] = (
    w_prophet * hybrid_eval["prophet"] +
    w_lastweek * hybrid_eval["lastweek"]
)

# Error
hybrid_eval["abs_error"] = (
    hybrid_eval["actual"] - hybrid_eval["forecast"]
).abs()

hybrid_mae = hybrid_eval["abs_error"].mean()

print(f"Hybrid MAE: {hybrid_mae:.2f}")
```

```

# Plot
plt.figure(figsize=(14,5))
hybrid_eval["actual"].plot(label="Actual Demand", color="black")
hybrid_eval["forecast"].plot(label="Hybrid Forecast", color="purple")

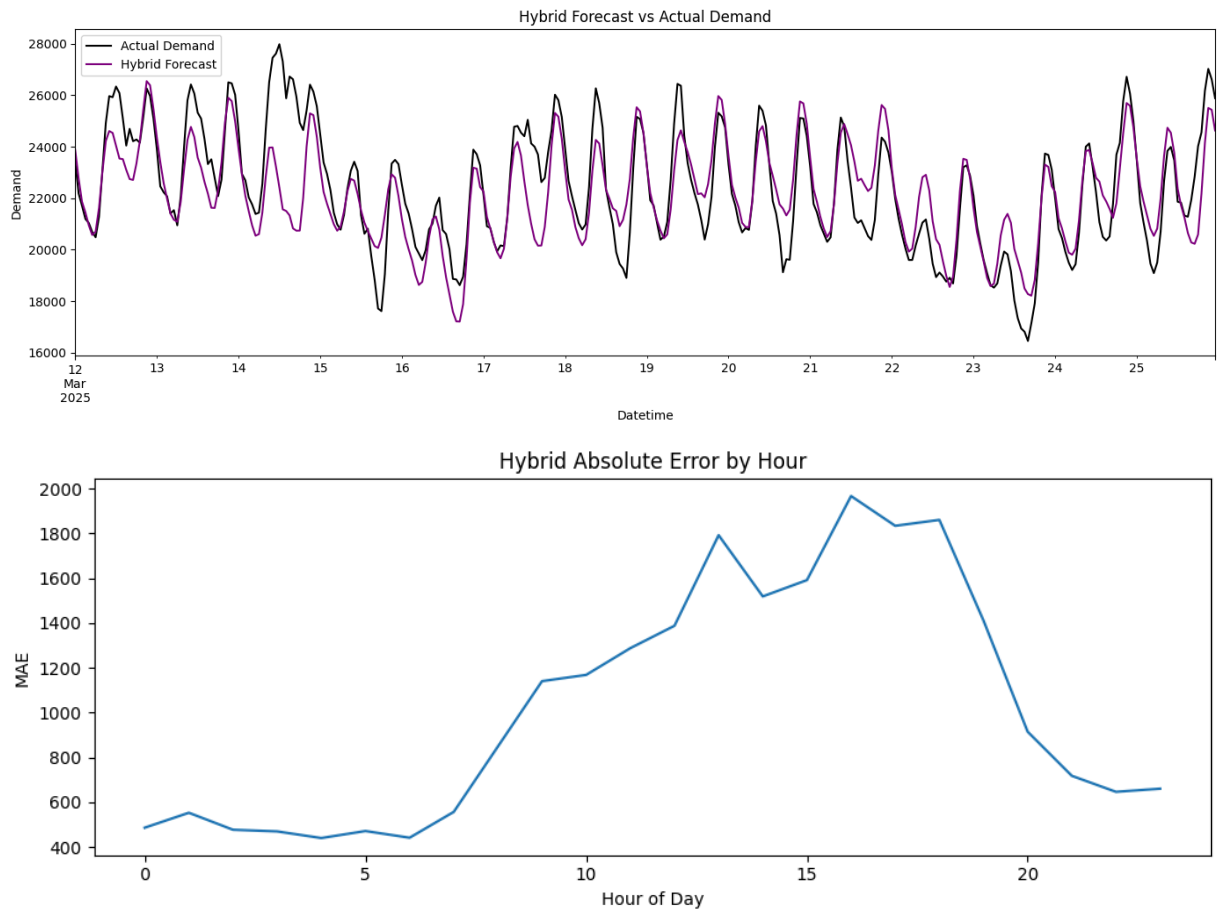
plt.title("Hybrid Forecast vs Actual Demand")
plt.xlabel("Datetime")
plt.ylabel("Demand")
plt.legend()
plt.tight_layout()
plt.show()

# Hourly MAE
plt.figure(figsize=(10,4))
hybrid_eval.groupby(hybrid_eval.index.hour)["abs_error"].mean().plot()

plt.title("Hybrid Absolute Error by Hour")
plt.xlabel("Hour of Day")
plt.ylabel("MAE")
plt.tight_layout()
plt.show()

```

Hybrid MAE: 1026.76



In [439...

```

# Model Scoreboard & Comparisons

# Rename forecast columns locally
yesterday_plot = baseline_eval.rename(columns={"forecast": "Yesterday"})

```

```

lastweek_plot = lastweek_eval.rename(columns={"forecast": "Last Week"})
sarima_plot   = sarima_eval.rename(columns={"forecast": "SARIMAX"})
prophet_plot  = prophet_eval.rename(columns={"forecast": "Prophet"})

# Compute MAE
yesterday_mae = yesterday_plot["abs_error"].mean()
lastweek_mae  = lastweek_plot["abs_error"].mean()
sarima_mae    = sarima_plot["abs_error"].mean()
prophet_mae   = prophet_plot["abs_error"].mean()
hybrid_mae    = hybrid_eval["abs_error"].mean()

scoreboard = pd.DataFrame({
    "Model": ["Yesterday", "Last Week", "SARIMAX", "Prophet", "Hybrid"],
    "MAE": [yesterday_mae, lastweek_mae, sarima_mae, prophet_mae, hybrid_mae]
})

# Improvement vs yesterday
scoreboard["Improvement vs Yesterday (%)"] = (
    (yesterday_mae - scoreboard["MAE"]) / yesterday_mae * 100
)

print(scoreboard.sort_values("MAE"))

# Visual 1 - Forecast comparison
plt.figure(figsize=(10,5))

scoreboard_sorted = scoreboard.sort_values("MAE")

plt.bar(scoreboard_sorted["Model"], scoreboard_sorted["MAE"])
plt.title("Model Accuracy Comparison (Lower MAE = Better)")
plt.ylabel("MAE")
plt.xlabel("Model")

plt.tight_layout()
plt.show()

# Visual 2 - % Improvement
plt.figure(figsize=(10,5))

plt.bar(scoreboard_sorted["Model"], scoreboard_sorted["Improvement vs Yesterday (%)"])
plt.title("Improvement vs Yesterday Baseline")
plt.ylabel("Improvement (%)")
plt.xlabel("Model")

plt.axhline(0) # shows who is worse than baseline
plt.tight_layout()
plt.show()

# Visual 3 - Hourly MAE
plt.figure(figsize=(12,5))

yesterday_plot.groupby(yesterday_plot.index.hour)["abs_error"].mean().plot(label="Y")
lastweek_plot.groupby(lastweek_plot.index.hour)["abs_error"].mean().plot(label="Las

```

```

sarima_plot.groupby(sarima_plot.index.hour)["abs_error"].mean().plot(label="SARIMAX")
prophet_plot.groupby(prophet_plot.index.hour)["abs_error"].mean().plot(label="Proph")
hybrid_eval.groupby(hybrid_eval.index.hour)["abs_error"].mean().plot(label="Hybrid")

plt.title("MAE by Hour of Day")
plt.xlabel("Hour")
plt.ylabel("MAE")
plt.legend()
plt.tight_layout()
plt.show()

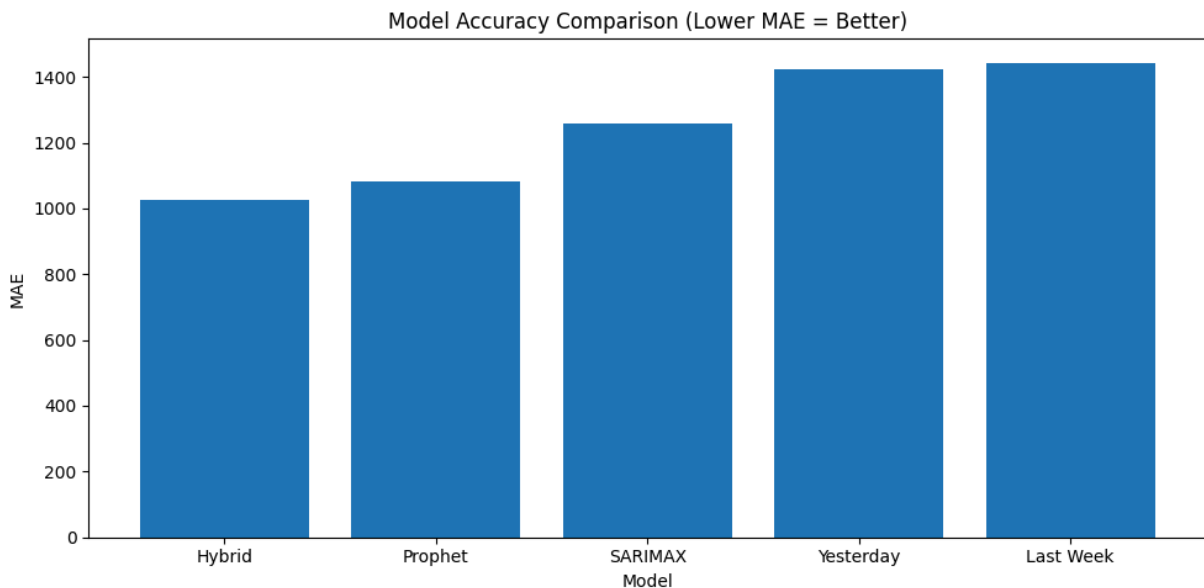
# Visual 4 - Forecast Comparisons
plt.figure(figsize=(15,6))

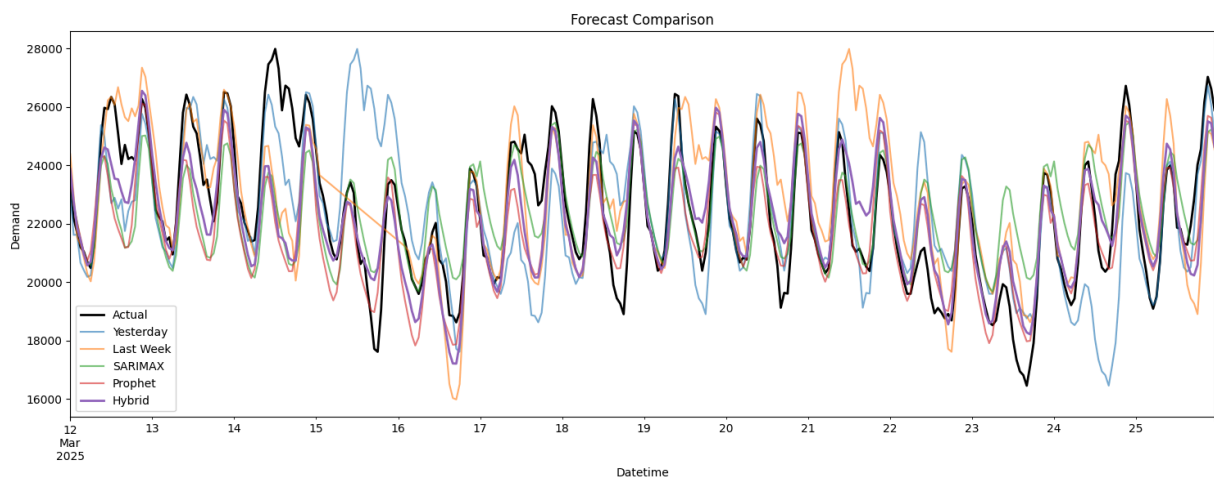
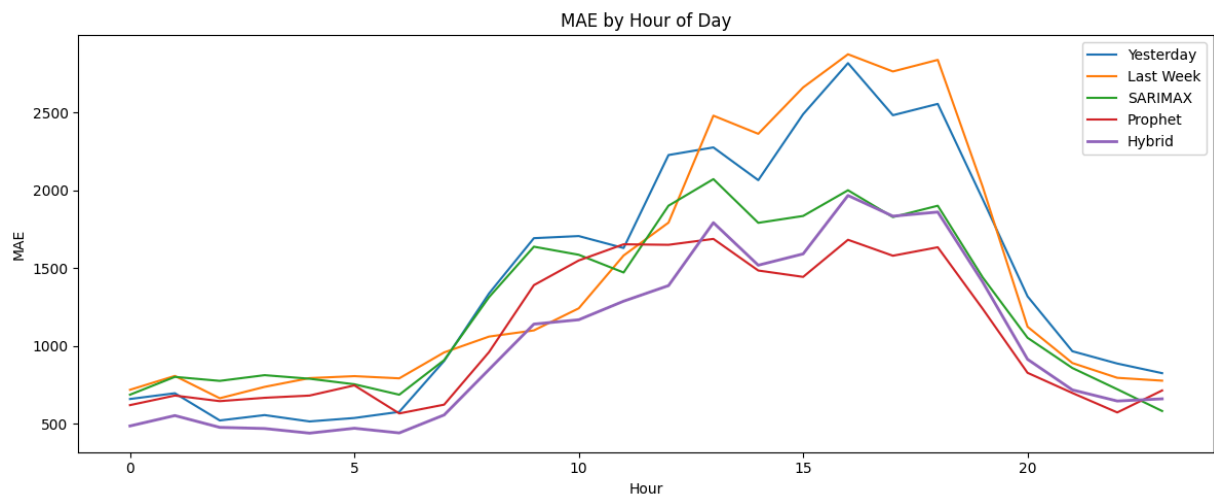
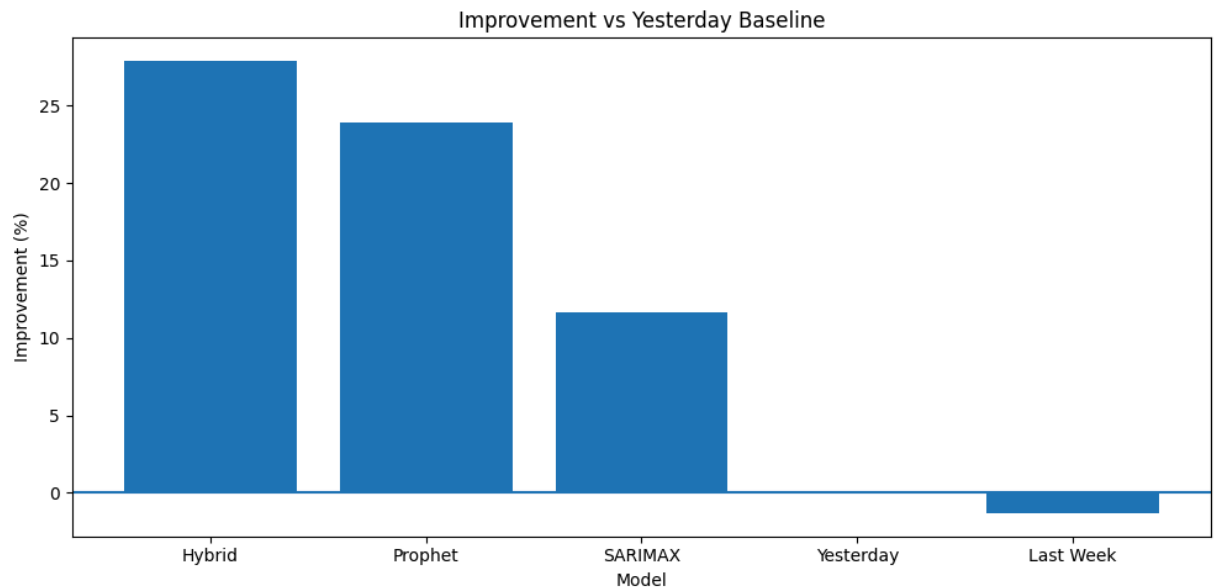
yesterday_plot["actual"].plot(label="Actual", color="black", linewidth=2)
yesterday_plot["Yesterday"].plot(alpha=0.6)
lastweek_plot["Last Week"].plot(alpha=0.6)
sarima_plot["SARIMAX"].plot(alpha=0.6)
prophet_plot["Prophet"].plot(alpha=0.6)
hybrid_eval["forecast"].plot(label="Hybrid", linewidth=2)

plt.title("Forecast Comparison")
plt.xlabel("Datetime")
plt.ylabel("Demand")
plt.legend()
plt.tight_layout()
plt.show()

```

	Model	MAE	Improvement vs Yesterday (%)
4	Hybrid	1026.758411	27.912300
3	Prophet	1083.345476	23.939378
2	SARIMAX	1258.765959	11.623278
0	Yesterday	1424.318452	0.000000
1	Last Week	1443.456786	-1.343684





Conclusion

In this project, I built and evaluated multiple approaches for forecasting hourly demand over a variable two-week period from January 1 2024 to December 31 2025. The objective was not simply to produce forecasts, but to understand how much performance improvement

more sophisticated time-series methods could deliver relative to simple and highly interpretable baselines.

Modeling Strategy

I began by implementing two naïve benchmarks:

- **Yesterday baseline** – assumes demand will match the same hour from the previous day.
- **Last-week baseline** – assumes demand will match the same hour one week earlier.

These references are critical because they are inexpensive, transparent, and often surprisingly strong. Any advanced model must beat them to justify added complexity.

From there, I developed more advanced forecasting systems:

- **SARIMAX**, incorporating autoregressive structure, daily seasonality, and calendar effects such as hour of day and day of week.
- **Prophet**, leveraging built-in daily and weekly seasonal components with flexible changepoints.
- A **hybrid approach** that blends model-based structure with recent observed behavior.

All data preparation, feature engineering, validation design, modeling, and diagnostics were completed independently.

Lessons Learned:

1. **Beating naïve forecasts is hard.** Simple methods set a very high bar.
2. **Recency matters.** The most recent patterns can outweigh complex historical relationships.
3. **Model combinations are powerful.** Blending approaches often yields more gain than tuning a single method.